Open Source Formal Verification with PSL

Coverage

Yann Thoma

Reconfigurable and Embedded Digital Systems Institute
Haute Ecole d'Ingénierie et de Gestion du Canton de Vaud



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported License

November 2026

1 Introduction

2 Code coverage

Coverage

- Coverage is the concept of knowing if we observed a certain number of sequences, states, or values
- Can be applied in:
  - Simulation
  - Formal verification

`cover` directive

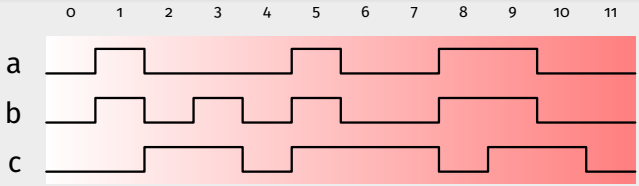
- It can be interesting to know if a certain sequence has been observed
- The operator `cover` is just what we need
- Example:
  - `cover {a;b;a} report "Seen";` displays a message when the sequence `a` followed by `b` and then again `a` is observed

`assert` and coverage

- SBY `cover` mode searches for reachable cover goals; it does not prove assertion consequents
- Temporal sequence implications generate cover goals for their antecedent sequences
- A plain Boolean implication using `->` does not generate this antecedent-coverage goal

Coverage - Example

Example



PSL

```
assert always ({a;c} |=> {c});
cover {a and b};
assert always (a -> b);
assert always (b |-> a);
assert always ({a} |-> {b});

cover_fail: cover {a;a;b};
seq_cover_fail: assert always ({a;a;b} |=> {c});
```

Not part of coverage

Part of coverage

Part of coverage

assert success but no coverage

assert and coverage

PSL

```
assert always ({a;c} |=> {c});
cover {a and b};
assert always (a -> b);
assert always (b |-> a);
assert always ({a} |-> {b});

cover_fail: cover {a;a;b};
seq_cover_fail: assert always ({a;a;b} |=> {c});
```



Not part of coverage

Part of coverage

Part of coverage

Vacuous property

- The temporal assertions generate goals for antecedents `{a;c}`, `b`, `{a}` and `{a;a;c}`
- The cover statement generate goals for `{a and b}`
- `(a -> b)` is retained as an assertion but creates no cover goal for `a`

assert and coverage

PSL

```
assert always ({a;c} |=> {c});
cover {a and b};
assert always (a -> b);      ← Not part of coverage
assert always (b |-> a);     ← Part of coverage
assert always ({a} |-> {b}); ← Part of coverage

cover_fail: cover {a;a;b};
seq_cover_fail: assert always ({a;a;b} |=> {c}); ← Vacuous property
```



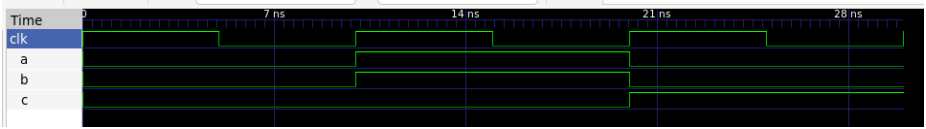
- Reaching an antecedent or cover goal does not show that the consequent holds
- Cover goals are most informative when inputs are constrained or the goal concerns meaningful design behavior
  - An unreached goal may indicate an unreachable behavior, an overly restrictive assumption, or an insufficient search depth
  - Coverage helps identify vacuous properties and untested behaviors

Coverage - Example output

- SBY `cover` mode reports reached and unreached cover goals
- Example output with all covered coverage goals:

```
SBY 16:18:33 [run_cover] engine_0: Status returned by engine: pass
SBY 16:18:33 [run_cover] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY 16:18:33 [run_cover] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
SBY 16:18:33 [run_cover] summary: engine_0 (smtbmc z3) returned pass
SBY 16:18:33 [run_cover] summary: cover trace: run_cover/engine_0/trace0.vcd
SBY 16:18:33 [run_cover] summary:   reached cover statement psl_sequence.\assert_success2_cover.cover at  step 1
SBY 16:18:33 [run_cover] summary:   reached cover statement psl_sequence.\assert_success3_cover.cover at  step 1
SBY 16:18:33 [run_cover] summary:   reached cover statement psl_sequence.cover_success at  step 1
SBY 16:18:33 [run_cover] summary: cover trace: run_cover/engine_0/trace1.vcd
SBY 16:18:33 [run_cover] summary:   reached cover statement psl_sequence.\assert_sequence_cover.cover at  step 2
SBY 16:18:33 [run_cover] DONE (PASS, rc=0)
```

- Trace for `assert always ({a;c} |=> {c})`:



Diagnosing an unreached antecedent

Request/acknowledge example

```
assume_no_req: assume always (not req);
assert_req_ack: assert always (req -> ack);
cover_req: cover {req};
```

- With this assumption, `req` can never occur, so `cover_req` is unreachable and the implication can pass vacuously
- This may expose an over-restrictive environment assumption; alternatively, the request may truly be unreachable in the modeled design
- Review the specification, assumptions, and design; also check whether the cover depth is sufficient
- If `req` is reached after revising the model, that exercises the antecedent but does not prove `ack`; prove the assertion separately

Double implication

PSL

```
assert always ({a;c} |=> ({d;d} |=> {b}));
```

- How to be sure we covered this assertion?

PSL

```
cover {a;c};
cover {d;d};
cover {a;c;d;d};
```

Not sufficient

Not sufficient

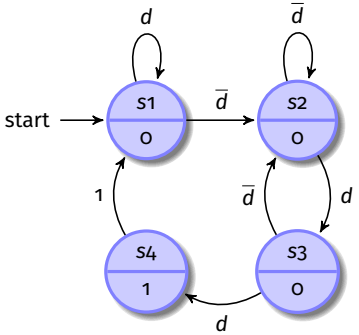
Coverage - Example output

- SBY `cover` mode reports reached and unreached cover goals
- Example output with uncovered coverage goals:

```
SBY 16:19:24 [run_cover] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY 16:19:24 [run_cover] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
SBY 16:19:24 [run_cover] summary: engine_0 (smtbmc z3) returned FAIL
SBY 16:19:24 [run_cover] summary: cover trace: run_cover/engine_0/trace0.vcd
SBY 16:19:24 [run_cover] summary:   reached cover statement psl_sequence.\assert_success2_cover.cover at  step 1
SBY 16:19:24 [run_cover] summary:   reached cover statement psl_sequence.\assert_success3_cover.cover at  step 1
SBY 16:19:24 [run_cover] summary:   reached cover statement psl_sequence.cover_success at  step 1
SBY 16:19:24 [run_cover] summary: cover trace: run_cover/engine_0/trace1.vcd
SBY 16:19:24 [run_cover] summary:   reached cover statement psl_sequence.\assert_sequence_cover.cover at  step 2
SBY 16:19:24 [run_cover] summary: unreached cover statements:
SBY 16:19:24 [run_cover] summary:   psl_sequence.\gen_fail.assert_sequence_cover_fail.cover at
SBY 16:19:24 [run_cover] summary:   psl_sequence.\gen_fail.cover_fail at
SBY 16:19:24 [run_cover] DONE (FAIL, rc=2)
SBY 16:19:24 The following tasks failed: ['cover']
```

State machine - transitions

- If we have access to the internal architecture
- Ensure coverage of expected transitions



Cover allowed transitions

```
cover {state = s1; state = s1};
cover {state = s1; state = s2};
cover {state = s2; state = s2};
cover {state = s2; state = s3};
cover {state = s3; state = s2};
cover {state = s3; state = s4};
cover {state = s4; state = s1};
```

Coverage - When?

- Run coverage after the relevant assertions have passed their `bmc` or `prove` checks; coverage is diagnostic, not a proof
- It checks whether selected behaviors or assertion antecedents are reachable in the current model
  - An unreached goal may be unreachable by design, excluded by an assumption, or beyond the configured cover depth
- With SBY:
  - `sby -yosys "yosys -m ghdl" -f run.sby cover`
  - Inspect the task's `logfile.txt` and final summary for reached goals and reported trace paths; label cover directives to identify them easily
  - Open a reported cover trace (typically under `<task>/engine_0/traceN.vcd`) in GTKWave to inspect how the behavior was reached

coverage and bmc/prove

- Like `bmc`, `cover` is depth-limited
- Coverage after a `bmc` run
  - Run it with the same depth (run faster)
  - The bmc run may have been too shallow (but PASS), and the coverage run can detect some properties that are not reachable at the bmc depth
- Coverage after a `prove` run
  - If the prover uses a depth limit, run coverage with the same depth (run faster)
  - If the prover does not use a depth limit, run coverage with a reasonable depth (run faster)
    - If it fails, increase the depth and rerun coverage

Coverage and prove

- Other tools embed coverage in the prove mode
  - A proof can end up *vacuous* if the antecedent is unreachable
  - The tool can report the antecedent as a cover goal, and if it is unreachable, the proof is vacuous
- Some tools can report the cover goals for all antecedents, even if the proof is not vacuous
  - This can be useful to check that all antecedents are reachable, even if the proof is not vacuous

A parenthesis for simulations

Coverage in simulation

- Cover statements are also useful in simulation
- Can drive random-based coverage-driven verification
- Can detect that some sequences of events have been observed during simulation

Code coverage (1)

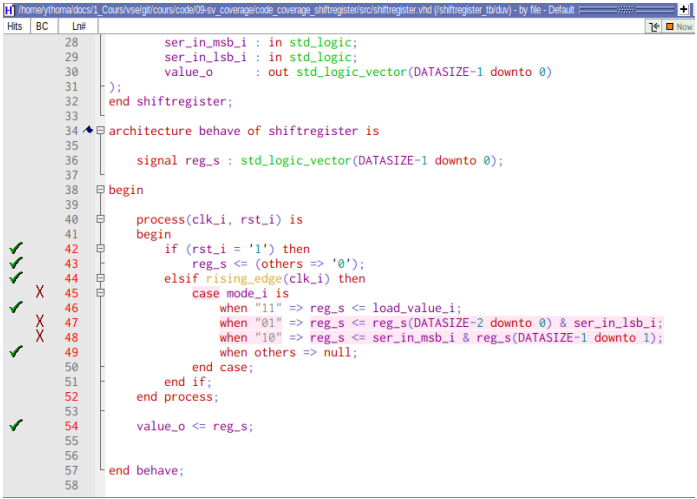
- In simulation, most simulators allow code coverage analysis
  - Have all the lines of code been executed?
  - Have all the conditional branches been observed in both choices?

Questasim

```
vcom +cover myfile.vhd
vsim -coverage work.myfile_tb
```

- Can be visualized in Questasim
- Allows to compute a global coverage over various scenarios and the Verification Run manager

Code coverage (2)



Code coverage (3)

- Goal:
  - 100% of code coverage on the design files
- Do not add the testbenches

```
if (observed /= expected) then
    report "Huge mistake";   <-- We expect this line not to be reached
end if;
```

- Except if you have testbenches with built-in self-tests
  - Need a way to inject errors in the observed values

Conclusion

- Coverage is key for ensuring verification is complete
  - Well, as complete as possible
- In simulation
  - Activate code coverage
  - Use functional coverage when possible (OSVVM or SV coverage)
- In formal
  - Check coverage to ensure the assertions can be observed
    - Do this when all assertions are checked
    - It does not imply you did check enough
    - There is no real way of being sure we had enough assertions
    - Actually if we implement all the possible properties we could synthesize a module based on the properties
    - Exhaustively specifying every possible property is impractical for real systems
    - Try to identify the relevant ones to complement your simulation-based verification